

Designed by Remus Teodorescu 2026

LECTURE 6.1

Loss Functions and Gradients

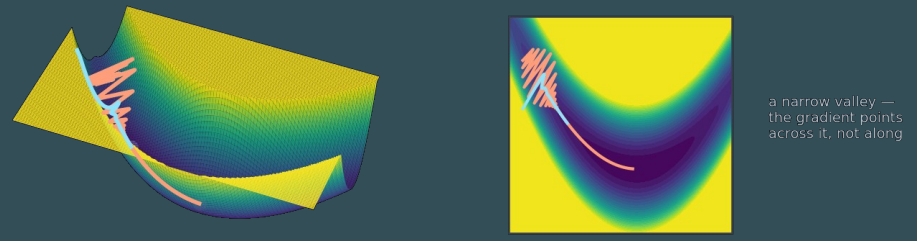
Remus Teodorescu

With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

LIKELIHOOD · DESCENT · BACKPROPAGATION



Every network in this course is trained the same way. The loss says how wrong the network is, and it follows from an assumption about the noise: squared error for Gaussian noise, cross entropy for classes. The gradient, computed by backpropagation, says which way to move, and an optimiser takes the step: plain descent zig-zags across the valley, Adam walks along it.

Learning Objectives

SECTIONS, IN ORDER

1. **Loss function** — where a loss comes from: squared error, cross entropy, the noise model
2. **Optimisers** — gradient descent and SGD, momentum, Adam, L-BFGS, the learning rate
3. **Gradients** — backpropagation, autograd, and where the weights start

One example runs through the lecture: a load cell read forty times, the data of Ex_06 notebook 01.

READ, AND WHAT EACH ANSWERS

UDL ch. 5	which loss, and why that one?
UDL ch. 6	how does a parameter move, and how far?
UDL ch. 7	where do the gradients come from, and where do the weights start?

Recap

1 – 2 · RECURRENT NETWORK

1 What is a recurrent network, and what function does it implement?

A network that reads a sequence one step at a time and keeps a hidden state as its memory. At each step the new state is computed from the input and the previous state, with the same weights, and a read-out gives an answer at the end or at every step. *GBC ch. 10.1 – 10.2; Ex_05 notebook 04*

2 How does it differ from a dense network, and what must a forecast beat?

One set of weights serves a sequence of any length, and the order of the samples is part of the input, where a dense network sees a fixed row. A forecast has to beat persistence, the guess that the next value equals the last one. *GBC ch. 10.2; Ex_05 notebook 04*

3 – 4 · LSTM

3 What is an LSTM, and what do its gates do?

A recurrent network with two memories: a cell state that is carried along and a hidden state that is shown. Three gates decide what to keep of the cell state, what to add to it and what to show. The forget bias starts at one, so the cell keeps by default. *GBC ch. 10.10; Ex_05 notebook 04*

4 Why does an LSTM remember longer than a plain recurrent network?

In a plain recurrent network the gradient is multiplied by the same weights at every step and vanishes over a long sequence. The LSTM's cell state is updated by addition, so the gradient passes along it nearly unchanged. In notebook 04 it had the lowest held-out error. *GBC ch. 10.7, 10.10; Ex_05 notebook 04*

Loss Function

Loss function — one number that says how badly the model's predictions match the measured outputs; training looks for the parameters that make it smallest. By maximum likelihood it is minus the log of the probability the model assigns to the measured outputs — in plain words, how surprised the model is by what you measured. That gives mean squared error for Gaussian noise (random errors, bell-shaped around the true value) and cross entropy for classification.

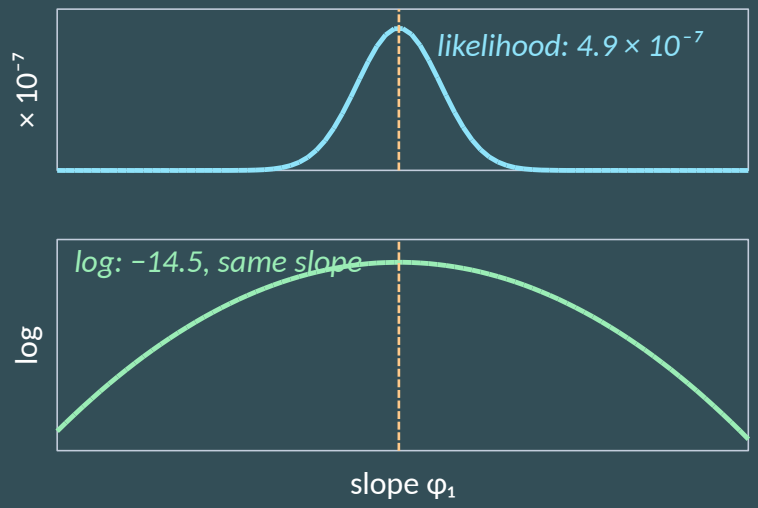
WHY MINUS THE LOG OF THE LIKELIHOOD

- **many small probabilities multiplied** — too small to store
- **the log turns the product into a sum** — easy to compute
- **the log only rises** — the best parameters stay the same

$$\hat{\theta} = \operatorname{argmin}_{\theta} \; -\sum_{i} \log P(y_i \mid f(x_i, \theta))$$

θ the parameters, y_i the measurements, $f(x_i, \theta)$ the prediction; minus, since optimisers minimise

THE LOG KEEPS THE MAXIMUM



NEGATIVE LOG LIKELIHOOD IN PYTORCH

```
mu, s = model(x).unbind(-1)      # mean and log-spread
dist = Normal(mu, s.exp())       # a Gaussian per input
loss = -dist.log_prob(y).sum()   # minus the log likelihood
loss.backward()                  # its gradients
```

Predicting a Distribution

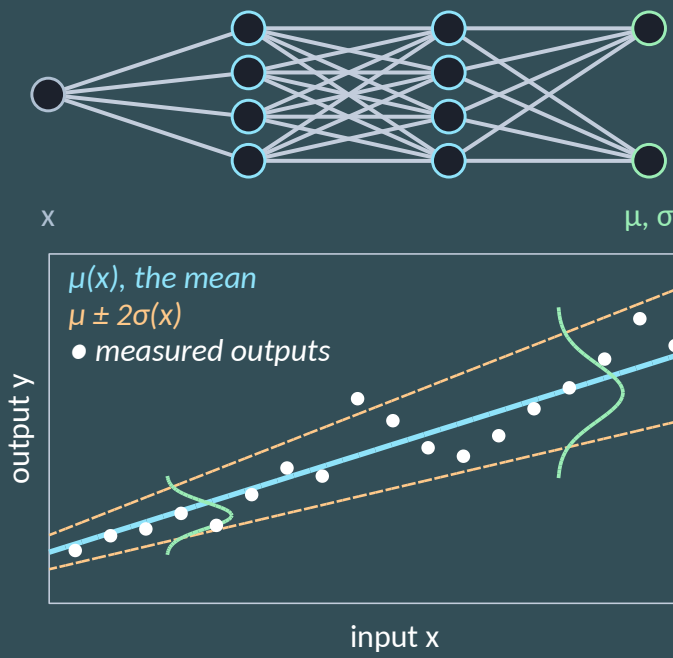
Predicting a distribution — instead of a single value, the network outputs the parameters of a probability distribution over the measured output: for Gaussian noise, the mean μ and the spread σ, each a function of the input x. The prediction is the most likely value, μ; σ says how far to trust it.

WHY A NETWORK, IF μ AND σ ARE EASY TO COMPUTE

- **many readings at one input** — mean and spread, by hand
- **one reading per input** — learn μ(x) and σ(x) instead
- **a straight line** — least squares, exactly (notebook 01)
- **curved, or many inputs** — the network earns its place

TALK · 2 MIN How do you understand a network that predicts a distribution rather than a single value? Then, in the plot on the right, at which input would you trust its prediction least, and why?

THE MEAN AND THE SPREAD



schematic: the spread grows with x, so σ is predicted too; a green bell is the distribution at one input.

Squared Error

Squared error — the loss that follows when every measured output is the model's prediction plus Gaussian noise. The network predicts the mean μ; the spread σ is a fixed constant. Minus the log of the Gaussian is, up to a constant, the sum of squared differences, so least squares is maximum likelihood.

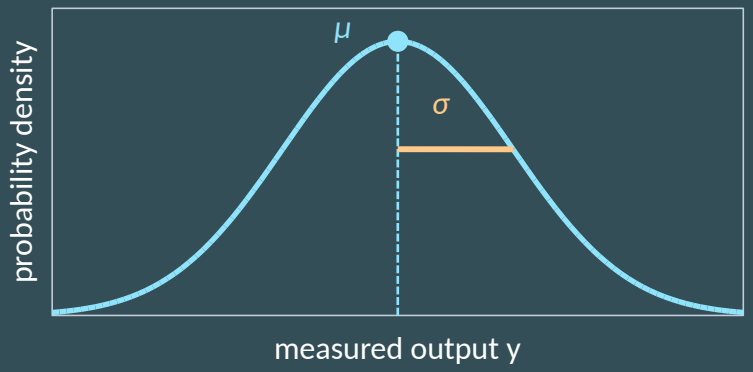
A LOAD CELL, READ FORTY TIMES

- **one reading** — the true value plus Gaussian noise
- **the network predicts μ and σ** — the centre and the spread
- **σ the same for every reading** — it drops out of the loss
- **Ex_06 notebook 01** — both fits give the same line

$$-\log P = \frac{1}{2\sigma^2}\sum_i (y_i - f(x_i, \theta))^2 + \text{const}$$

y_i the measurement, $f(x_i)$ the prediction, σ the noise level — constant, so it drops out

ONE READING, AS A DISTRIBUTION



μ places the bell, σ sets its width. With σ fixed, raising the bell at your readings is the same as shrinking the squared error.

SQUARED ERROR IN PYTORCH

```
loss = ((model(x) - y) ** 2).mean()   # squared error, by hand
loss = nn.MSELoss()(model(x), y)      # the same, built in
```

Cross Entropy

Cross entropy — the loss for classification. A sigmoid (yes or no) or a softmax (one of K) turns the network's outputs into probabilities, and the loss is minus the log of the probability given to the observed class: the same maximum-likelihood recipe as squared error, with a Bernoulli or categorical noise model instead of a Gaussian.

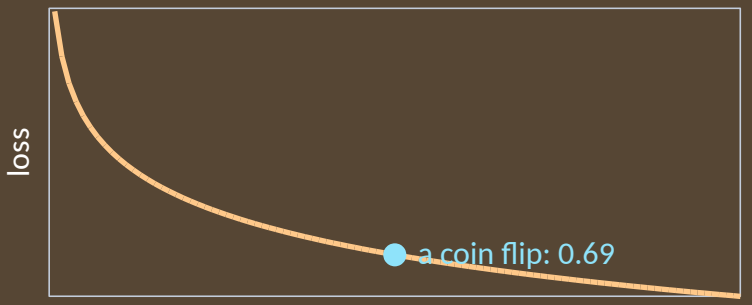
YES OR NO, AND ONE OF K CLASSES

- **Bernoulli** — yes or no: 1 with probability p, 0 with 1 − p
- **categorical** — one of K, probabilities that sum to one
- **confident and wrong** — charged heavily, by the log
- **the check** — an untrained K-class model reports about ln K

$$\mathcal{L} = -\sum_i [y_i \log p_i + (1 - y_i)\log(1 - p_i)]$$

$y_i \in \{0, 1\}$ the label, p_i the predicted probability of class 1

THE PENALTY, AND THE SOFTMAX



$$p_k = \frac{\exp(z_k)}{\sum_j \exp(z_j)}$$

z_k the logit, the raw score for class k, p_k its probability

CROSS ENTROPY IN PYTORCH

```
z = model(x)                          # logits: raw scores
loss = nn.BCEWithLogitsLoss()(z, y)   # sigmoid inside
loss = nn.CrossEntropyLoss()(z, k)    # softmax inside
```

Noise Model

Noise model — the distribution assumed for the difference between a measurement and the underlying value. The loss follows from it: Gaussian noise gives squared error; Laplace noise, which makes large errors less rare, gives absolute error, so a single outlier has far less influence; for classification the noise model is Bernoulli or categorical and the loss is cross entropy.

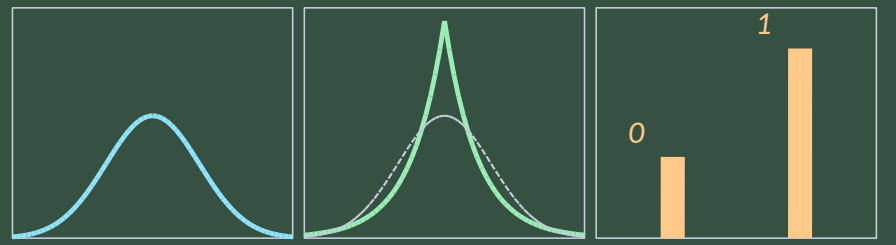
ONE BAD READING IN FORTY

A Gaussian distribution gives a far-out reading almost no probability, so the fit moves towards it, and the squared penalty grows with the square of the distance.

In Ex_06 notebook 01 one bad reading moves the least-squares slope by nine per cent and the intercept by thirty-five; the absolute-error fit stays within two per cent of the truth.

Absolute error has no derivative at zero. Say which noise model you assumed, in the report, in one sentence.

THREE NOISE MODELS, THREE LOSSES



noise model	it says	the loss
Gaussian	large errors rare	squared
Laplace	large errors less rare	absolute
Bernoulli	yes or no, with p	cross entropy
categorical	one of K	cross entropy

Dashed: the Gaussian of the same spread, for comparison.

Gradient Descent

Gradient descent — an iterative method for finding low values of the loss: compute the gradient, the direction in which the loss rises fastest, and take a small step the other way, its size set by the learning rate. Every step uses the whole training set, so the path is smooth and each step is as expensive as a pass over the data.

DOWNHILL IN SMALL STEPS

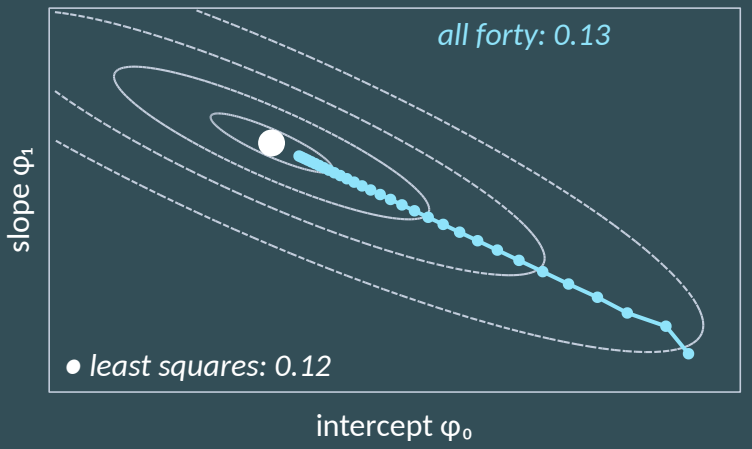
- **long strides where it is steep** — short near the bottom
- **every step** — the whole training set

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta \mathcal{L}(\theta_t)$$

θ the parameters, α the learning rate, $\nabla_\theta \mathcal{L}$ the gradient, t the step

TALK · 2 MIN How would you explain gradient descent to someone who has never met it? Then, in the plot on the right, why do the steps shrink as the path nears the white dot?

NOTEBOOK 01'S LINE FIT, FROM ABOVE



loss after thirty steps; dashed rings are equal loss

GRADIENT DESCENT IN PYTORCH

```
opt = torch.optim.SGD(model.parameters(), lr=0.3)  # α
for step in range(30):                  # thirty steps
    loss = loss_fn(model(X), Y)         # loss on all forty
    opt.zero_grad(); loss.backward() # its gradient
    opt.step()                          # θ ← θ − α∇L
```

Stochastic Gradient Descent

Stochastic gradient descent — gradient descent in which each step uses the gradient computed on a small random batch of the training data instead of the whole set. Each step is much cheaper and slightly noisy; the noise costs some precision near the minimum but helps escape poor minima and saddle points.

ONE BATCH PER STEP

- **one pass over all the batches** — an epoch
- **batch size** — not only a memory setting

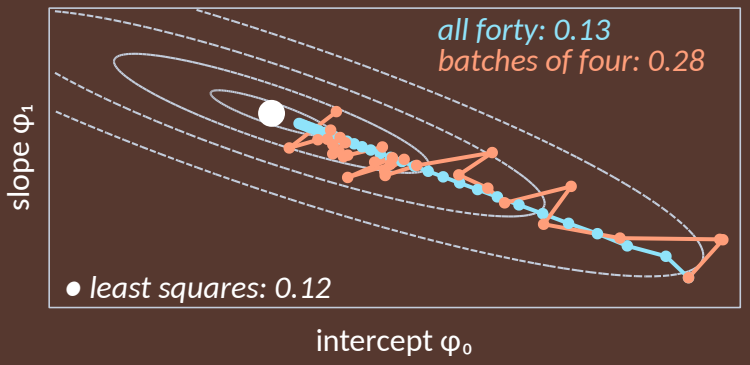
$$\theta_{t+1} = \theta_t - \alpha \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_\theta \ell_i(\theta_t)$$

B_t a random batch, $|B_t|$ its size, ℓ_i the loss on one example

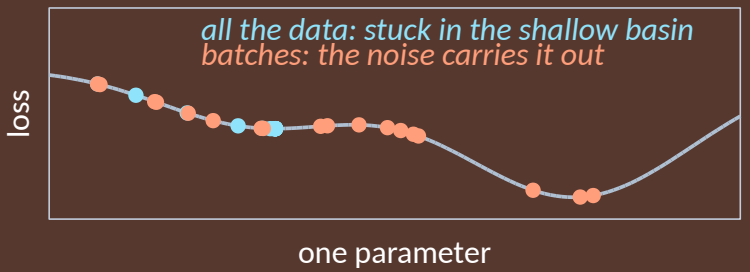
SGD IN PYTORCH

```
opt = torch.optim.SGD(model.parameters(), lr=0.25)  # α
for x, y in DataLoader(data, 4, shuffle=True): # random batches of four
    loss_fn(model(x), y).backward()  # the gradient on one batch
    opt.step(); opt.zero_grad()      # θ ← θ − α∇L, then clear it
```

ALL FORTY AGAINST BATCHES OF FOUR



A POOR MINIMUM, SCHEMATIC



Momentum

Momentum — a change to gradient descent in which each step follows an exponentially weighted average of the current and past gradients rather than the current gradient alone. In a narrow valley the components across the valley cancel and the components along it add up, so progress is faster and zigzags less.

A RUNNING AVERAGE OF THE GRADIENT

- **loss surfaces** — far steeper in some directions than others
- **a ball with mass** — not a point

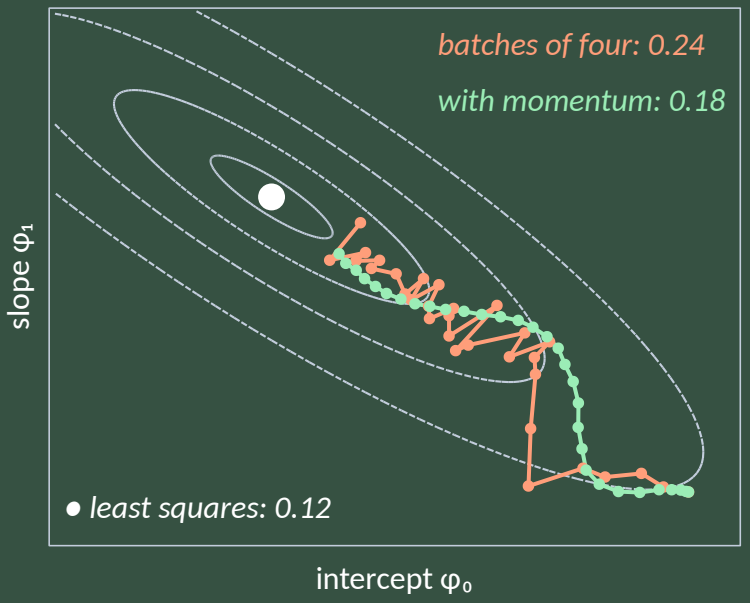
$$m_{t+1} = \beta m_t + (1-\beta)\nabla_\theta \mathcal{L} \quad \theta_{t+1} = \theta_t - \alpha\, m_{t+1}$$

m the running average of the gradient, β how much of it survives each step

MOMENTUM IN PYTORCH

```
opt = torch.optim.SGD(model.parameters(),  # gradient descent
                      lr=0.2,              # the step size α
                      momentum=0.9)        # β, how much of the past step stays
```

THE SAME BATCHES, WITH AND WITHOUT



$\alpha = 0.2, \beta = 0.9$, loss after thirty steps: momentum smooths the path and reaches the lower loss.

Adam

Adam — (adaptive moment estimation) gradient descent with momentum plus per-parameter scaling. It keeps exponentially weighted averages of the gradient and of its square and steps by one divided by the square root of the other, so every parameter moves at a similar rate whatever the size of its gradient, and the learning rate needs little tuning. The usual default.

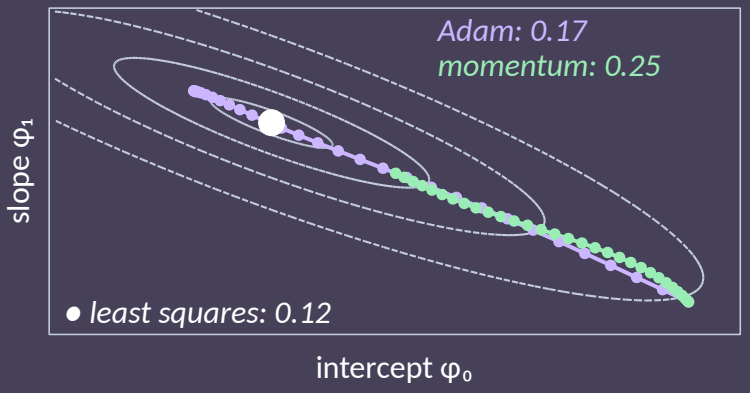
A LEARNING RATE PER PARAMETER

- **momentum fixed the direction** — Adam fixes the scale
- **small gradients still move** — divided by their own size
- **not monotone** — report the final loss or the best, and say which

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

m the averaged gradient, v its averaged square, ε stops a divide by zero

THE SAME STEP SIZE, THIRTY STEPS



$\alpha = 0.15$ for both. Adam's steps are about α long; momentum's are α times the gradient.

ADAM HYPERPARAMETRIZATION IN PYTORCH

```
opt = torch.optim.Adam(          # the optimiser
    model.parameters(),          # every weight
    lr=1e-3,                     # the one to tune
    betas=(0.9, 0.999))          # the two averages
opt.step(); opt.zero_grad()      # step, then clear
```

L-BFGS

L-BFGS — (limited-memory Broyden–Fletcher–Goldfarb–Shanno) a quasi-Newton method: it estimates the curvature of the loss from the last few gradients and uses it to choose the direction and the length of each step. Every step uses the full data set, and several evaluations of the loss.

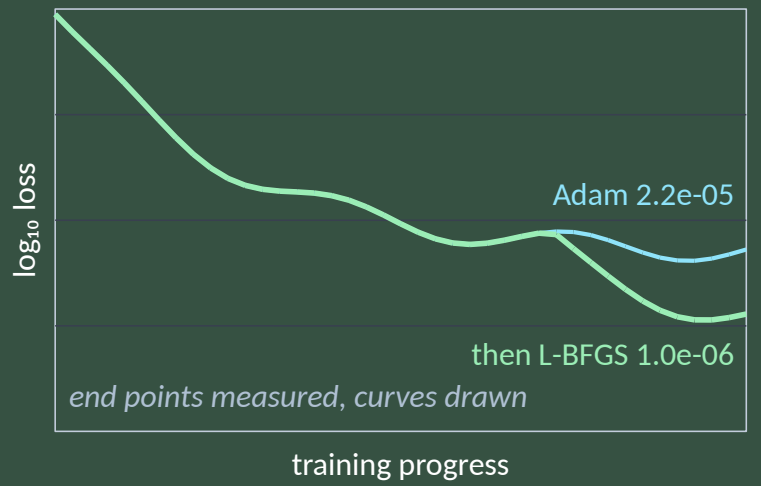
ADAM AGAINST L-BFGS

	Adam	L-BFGS
uses	the slope, averaged	slope and curvature
data per step	a batch	all of it
cost per step	one gradient	several evaluations
best	far from the minimum	near it, on a smooth loss

L-BFGS IN PYTORCH

```
opt = torch.optim.LBFGS(model.parameters())  # full batch
def closure():                    # called several times per step
    opt.zero_grad()               # clear old gradients
    loss = loss_fn(model(X), Y)   # the loss on all the data
    loss.backward(); return loss  # its gradient, and the loss
opt.step(closure)                 # one L-BFGS step
```

EX_06 NOTEBOOK 02: FINAL LOSSES



Adam, 2000 steps	into the right basin
then L-BFGS, 60	several hundred evaluations
gain	22 × over Adam alone

Learning Rate

Learning rate — the step size, how far the parameters move along the negative gradient at each step. Too large and training diverges or oscillates, too small and it crawls; it is often decreased during training. With the batch size and the momentum coefficients it is a hyperparameter of the training, set by hand or by search rather than learned.

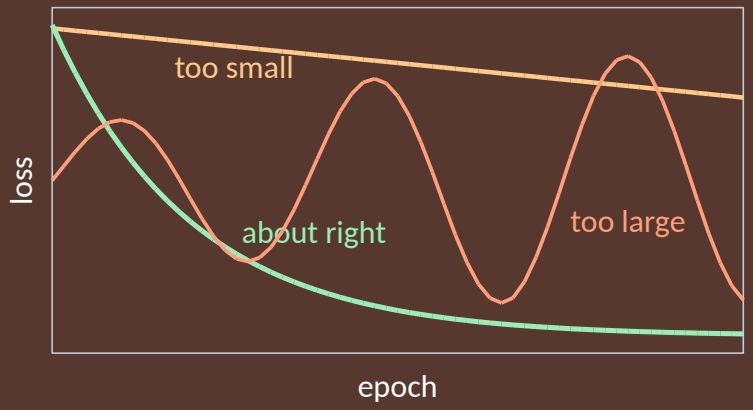
THE HYPERPARAMETER TO TUNE FIRST

- **search on a log scale** — factors of ten, not decimals
- **a decay schedule** — travel early, settle late
- **too small looks like a working run** — the costly failure

LEARNING RATE AND SCHEDULE IN PYTORCH

```
opt = torch.optim.Adam(model.parameters(), lr=1e-3)     # the step size
sched = torch.optim.lr_scheduler.StepLR(opt, 100, 0.5)  # halve every 100
```

THREE RATES, ONE SURFACE



WHAT TO TUNE, IN ORDER

1. learning rate — decades, not decimals
2. how long you train
3. batch size, then the rest

Backpropagation

Backpropagation — the algorithm that computes the derivative of the loss with respect to every parameter. The forward pass computes and stores every intermediate value; the backward pass applies the chain rule from the loss back through the network, reusing each partial product, so all derivatives cost about as much as one extra forward pass. Frameworks do this automatically (automatic differentiation, or autograd).

THE CHAIN RULE, IN THE ORDER THAT PERMITS REUSE

- **reuse** — every parameter shares the same partial products
- **the memory** — training stores what inference does not

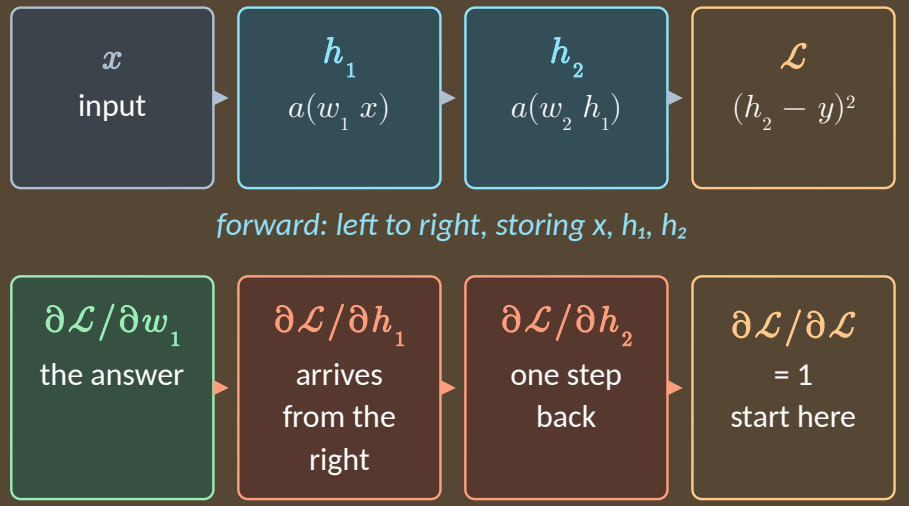
$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial h_2}\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial w_1}$$

w_1 a weight in the first layer, each factor is one layer, multiplied right to left

THE TWO PASSES IN PYTORCH

```
loss = loss_fn(model(x), y)   # forward: every value stored
loss.backward()               # backward: every gradient, one pass
model[0].weight.grad          # ∂L/∂w for the first layer
```

FORWARD, THEN BACKWARD



local × incoming	this weight's gradient
Ex_02 notebook 02	by hand, vs autograd

Autograd

Autograd — PyTorch's automatic differentiation. Every operation on a tensor that requires gradients is recorded while the forward pass runs, and backward() walks that record in reverse to compute the gradients; backpropagation is never written by hand. The same call, applied repeatedly, gives the higher derivatives needed for physics residuals.

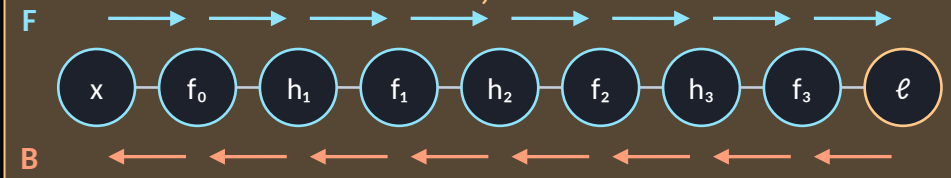
FOUR CALLS TO GET RIGHT

zero_grad()	gradients add up if you forget
backward() once	the record is freed after it
no_grad()	for inference — saves the memory
detach()	cuts the record deliberately

A FIRST AND A SECOND DERIVATIVE

```
x = torch.tensor(2.0, requires_grad=True)           # record x
y = x ** 3                                          # the forward pass
dy, = torch.autograd.grad(y, x, create_graph=True)  # 3x² = 12
d2y, = torch.autograd.grad(dy, x)                   # 6x = 12
```

UDL'S TOY NETWORK, BOTH WAYS



$$f_0 = \beta_0 + \omega_0 x, \quad h_1 = \sin f_0$$

$$f_1 = \beta_1 + \omega_1 h_1, \quad h_2 = \exp f_1$$

$$f_2 = \beta_2 + \omega_2 h_2, \quad h_3 = \cos f_2$$

$$f_3 = \beta_3 + \omega_3 h_3, \quad \ell = (f_3 - y)^2$$

$$\frac{\partial \ell}{\partial f_3} = 2(f_3 - y), \quad \frac{\partial \ell}{\partial h_3} = \omega_3 \frac{\partial \ell}{\partial f_3}$$

$$\frac{\partial \ell}{\partial f_2} = -\sin f_2 \frac{\partial \ell}{\partial h_3}, \quad \frac{\partial \ell}{\partial \omega_3} = h_3 \frac{\partial \ell}{\partial f_3}$$

β, ω biases and weights, f, h before and after each function

Forward stores every f and h; backward starts at the loss and multiplies, right to left, by each node's own derivative.

Initialisation

Initialisation — the parameters' values before training starts. Weights are random with a variance set by the layer's width, so that activations and gradients keep their scale from layer to layer instead of exploding or vanishing; biases start at zero.

WHERE THE PARAMETERS START, AND WITH WHAT SPREAD

- **all zero** — every neuron computes the same thing, for ever
- **too large or too small** — the activations grow, or die
- **PyTorch does this by default** — it matters in a layer you write

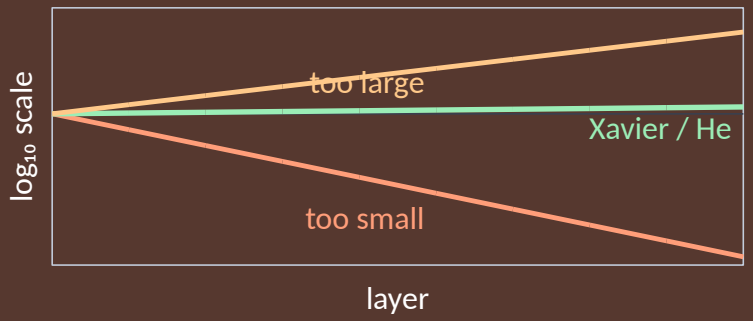
$$\mathrm{Var}(w) = \frac{2}{D_{\text{in}}} \qquad \mathrm{Var}(w) = \frac{2}{D_{\text{in}} + D_{\text{out}}}$$

w a weight, D_{in}, D_{out} the layer's inputs and outputs; first He, then Xavier

INITIALISATION IN PYTORCH

```
nn.init.xavier_normal_(layer.weight)                          # Glorot: tanh
nn.init.kaiming_normal_(layer.weight, nonlinearity='relu')    # He: ReLU
nn.init.zeros_(layer.bias)                                    # biases at zero
```

ACTIVATION SCALE, TEN LAYERS



THE TWO RULES

Xavier (Glorot)	variance 1 / fan-in, or 2 / (fan-in + fan-out): tanh, sigmoid
He (Kaiming)	variance 2 / fan-in, as ReLU zeroes half: the ReLU default

Advanced Topics

RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

topic	what it adds	read	where it makes a difference
quantile loss	a value exceeded only 10 % of the time	UDL ch. 5 notes	reserve and risk planning
focal loss	weights the rare class up	UDL ch. 5 notes	rare faults, rare events
AdaGrad, RMSProp	adaptive rates, before Adam	UDL ch. 6 notes	sparse data, recommenders
AdamW, warm-up	decoupled decay, a gentle start	UDL ch. 6 notes	large language models
line search	chooses each step's length	UDL ch. 6 notes	classical optimisation
checkpointing	stores fewer activations	UDL ch. 7 notes	large models on one GPU

WHY THEY ARE LEFT OUT

Each is a refinement of today's pieces, and most pay off at large scale. Squared error, cross entropy, Adam, L-BFGS and backpropagation train every network in this course.

EXERCISE

Exercise 6

THE NOTEBOOKS, IN ORDER

- **00** — checks the setup, and the closure L-BFGS needs
- **01** — a load cell read forty times: least squares from a Gaussian, one bad reading, then cross entropy
- **02** — the recipe of Part 2: Adam, then L-BFGS, each alone and chained
- **03 and 04** — after L6.2: transfer, and a battery that learns to trade
- **05** — the report: every notebook's answers, gathered, and one question to conclude

THE QUESTION EACH NOTEBOOK ANSWERS

01	which loss, and why that one?
02	why Adam first, then L-BFGS?

REPORT QUESTION, TO CONCLUDE

Across the notebooks: which one choice — the loss, the optimiser or the starting weights — changed your result most, and what number shows it?

SUMMARY

Key Takeaways

LOSS FUNCTION

what it is	one number: minus the log likelihood of the data
function	outputs and readings in; the net predicts μ, σ or p
train, tune	choose the distribution; the recipe gives the rest
vs squared	a Laplace tail: 2 % off where squares moved 35 %
used for	every fit: squared error, cross entropy, absolute

GRADIENT TRAINING

what it is	downhill steps; gradients by backpropagation
function	parameters in, a lower loss out, step after step
train, tune	learning rate first; Adam, then L-BFGS to finish
vs plain GD	Adam scales each step; L-BFGS adds curvature
used for	every network in this course, Part 2's too

Next — L6.2: training philosophies, from pre-training to reinforcement.

Questions

1 – 2 · LOSS FUNCTION

1 **What is a loss function, and where does it come from?**

One number that says how badly the predictions match the measured outputs. It is minus the log of the probability the model gives to the data: choose a distribution for the noise, let the network predict its parameters, and the loss follows. *UDL ch. 5.1 – 5.2*

2 **What does squared error assume, and which loss is used when that does not hold?**

That the noise is Gaussian with a fixed spread. If large errors are not rare, a Laplace distribution gives the absolute error: one wrong reading in forty moved it by 2 % where squared error moved by 35 %. For classes, the same recipe gives cross entropy. *UDL ch. 5.3 – 5.7; Ex_06 notebook 01*

3 – 4 · GRADIENT TRAINING

3 **What is gradient descent, and where do the gradients come from?**

Repeated small steps of the parameters in the direction that lowers the loss. Backpropagation gives the gradient of the loss with respect to every parameter in one forward and one backward pass, at about the cost of one extra pass, and autograd writes it for you. *UDL ch. 6.1, 7.2 – 7.4*

4 **How is training tuned, and how do SGD, Adam and L-BFGS differ from plain gradient descent?**

The learning rate comes first. SGD computes the gradient on a random batch, Adam scales each parameter's step by its own gradient history, and L-BFGS adds curvature. In notebook 02, Adam followed by L-BFGS ended about twenty times below Adam alone. *UDL ch. 6.2 – 6.5; Ex_06 notebook 02*

NEXT

L6.2 — Training Philosophies

Training a model from scratch is now the unusual case. Next time: starting from somebody else's weights, what pre-training and fine-tuning mean for a 2026 model, and how reinforcement learning trains one without labelled answers.

BEFORE THE NEXT SESSION

read	UDL ch. 9.3 — transfer learning
run	Ex_06 notebooks 00 to 02
bring	your four-optimiser loss plot
answer	the questions at the end of 01 and 02

Bring the plot. The gap between the SGD curve and the Adam curve is section two of today, measured.